



University of  
St Andrews

## **Conjure Oxide Book sections**

Brief review of the work done on the SAT subproject

Student ID: 230014406

Starting Semester I 2025-26

## Contents

|          |                                |          |
|----------|--------------------------------|----------|
| <b>1</b> | <b>Rule Sets</b>               | <b>3</b> |
| 1.1      | Representation Rules . . . . . | 3        |
| <b>2</b> | <b>Types of SAT Encodings</b>  | <b>4</b> |
| 2.1      | Logarithmic Encoding . . . . . | 4        |
| 2.2      | Direct Encoding . . . . .      | 4        |
| 2.3      | Order Encoding . . . . .       | 4        |
| <b>3</b> | <b>Functional Rust</b>         | <b>5</b> |

## I Rule Sets

While developing rule-based transformations for conjure oxide, it is useful to understand the structure of the rulesets and the ‘types’ of rules that can be used in conjure oxide. Let us first look at how rules actually function, not programmatically, but in an abstract sense. An understanding of (and some experience of) functional programming is incredibly helpful<sup>1</sup>. Also useful is an understanding of the idea behind graph machines<sup>2</sup> and an understanding of the difference between function results and side effects<sup>3</sup>.

In conjure oxide, the rules are functions that take in the expression tree and the symbol table as arguments and return a function result<sup>4</sup>, meaning that the original expression tree and symbol table are only modified through side effects of ‘rule’ functions<sup>5</sup>.

There are quite a few advantages to this system, including a few small quality of life things such as the ability to write more descriptive errors and to pattern match on function results. The most significant, however, is the ability to express rules as functions that can do whatever they need to do as long as they return a failure or a success. This means that an application failure can be treated as a recoverable result rather than a crash. Each function is self contained<sup>6</sup> meaning that the only things preserved are the initial Symbol Table and the Expression, along with the result being passed along the call stack. The most significant, however, is that this allows for code to be written in a way that enforces that the errors be handled. This is quite useful in general because it means that the codebase can leverage Rust’s type system to eliminate edge cases everywhere. The rule application is able to use it to avoid a situation where the rule engine fails unexpectedly and loses a large amount of work, or worse, a situation where a rule seems to have been applied and is visible in the trace but has not actually affected the tree because it failed<sup>7</sup>.

Now that we know how they are called, we can move on to how the rules themselves are designed. Broadly, there are two categories that we can divide the rules into: Transformation Rules and Representation rules. This is because despite being applied in the same ‘step’ in the process, they have different purposes from each other, but all of the rules of either type all share a common purpose.

### I.1 Representation Rules

Essence (which is the input language of conjure-oxide) defines domains which are not present in the type systems of the different solvers’ input languages, meaning they need to be encoded in some way into the input languages. The encoding is done using *representation rules*. They are a ‘type’ of rule which implements shared behaviour using a trait<sup>8</sup>. The representation rules must change essence expressions into the target solver’s input language while preserving all of the relevant information.

---

<sup>1</sup>For this, I am partial to Graham Hutton’s excellent *Programming in Haskell (2nd Ed)*. Also take a look at *Learn You a Haskell for Great Good!* by Miran Lipovača, especially if you prefer a lighter, more witty read.

<sup>2</sup>Take a look at this blogpost <https://amelia.how/posts/the-gmachine-in-detail.html> for a good explanation of graph machines (and bonus on lazy evaluation!)

<sup>3</sup>I first gained an understanding of this from the very source – Alonzo Church’s Paper *The Calculi of Lambda Conversion*, which has a very thorough explanation of the concept. However, for a far more accessible explanation is that a Function is a mapping from some input to some output. Anything that the function ‘permanently’ changes in the world outside its scope is a side effect

<sup>4</sup>`std::Result<T, E>` in rust records either a Success (`Ok(T)`) or a Failure (`Err(E)`). This type of ‘error reading’ is a functional concept – much like the `Either` Monad in Haskell. This is different from throwing exceptions like in imperative languages

<sup>5</sup>Worth noting that this is particularly nice in rust because of the ownership system and the error system

<sup>6</sup>Owing to the Ownership system in Rust, any extra memory allocated in the function is freed as soon as the function exits

<sup>7</sup>As long as a program doesn’t just call `unwrap` on a rule result... But if you do make this error, you can take some solace in knowing that cloudflare beat you to it!

<sup>8</sup>For details, check out the documentation

## 2 Types of SAT Encodings

There are three different types of SAT Encodings planned in conjure oxide. Of these, only Logarithmic Encodings have been implemented thus far. The three types are these:

- Direct Encodings
- Logarithmic Encodings
- Order Encodings

Each type of encoding has pros and cons, and a different one may be selected based on the type of constraint problem.

### 2.1 Logarithmic Encoding

The base principle is quite simple: encode an integer as a bitvector. This allows us to represent integers as a series of boolean constraints – one for each bit.

For example, the integer ‘6’ can be represented in binary as ‘0110’. We can then represent this as ( $P = 0 \vee 1, Q = 0 \vee 1, R = 0 \vee 1, S = 0 \vee 1$ ). The connection that is missing, however, is that this isn’t actually a representation of a constraint problem, but of a solution to a problem.

### 2.2 Direct Encoding

Direct encodings are the most straightforward type of encoding – it involves creating a vector of boolean variables, one corresponding to each member of the domain. Only one of these variables can be true at a time, and it is the one corresponding to the value of the integer.

### 2.3 Order Encoding

Order follows the same principle as direct encoding, but instead of each boolean variable ‘specifying’ a value in the way that direct encodings do, each bit specifies whether the integer corresponding to it is less than or equal to the integer variable’s value.

### 3 Functional Rust

Consider the following oft-quoted statement about Rust:

*Rust is blazingly fast and memory-efficient: with no runtime or garbage collector. Rust's rich type system and ownership model guarantee memory-safety and thread-safety — enabling you to eliminate many classes of bugs at compile-time.*

Taken specifically from the Rust Foundation's<sup>9</sup> page talking about why rust is a good language to use.

But let us focus on the implications of this statement on conjure oxide specifically. The key details that one needs to know here are that despite arguably being imperative, Rust adopts a lot of functional programming concepts in its design<sup>10</sup>. This is significant because the conjure oxide codebase uses a lot of these functional programming concepts.

Why do this? Making use of these allows for the codebase to use the Rust type system to eliminate failing cases using the code style and make the code more robust. It also makes the code far easier to write, because Rust's safety system enforces the return type of functions. By using the `Result` type, functions can explicitly state failure or success. It also allows for all functions to call each other linearly in a safe manner<sup>11</sup>.

This allows for the code to be written in such a way that the functions can do essentially whatever they need to do as long as they record the success/failure result. The error type allows for error propagation in a more sophisticated manner than exit codes, while still being more efficient than exceptions.

---

<sup>9</sup>Who are, of course, an entirely unbiased source

<sup>10</sup>Rust's original development language was OCaml, which is functional. The original developers adopted quite a few functional programming concepts from OCaml

<sup>11</sup>Think of this as calling functions in the style that one might in a loosey-goosey language like python without any enforced typing, except with enforced types